

Document number: P0017R0
Date: 2015-07-13
Project: Programming Language C++, Language Evolution Working Group
Reply-to: Oleg Smolsky <oleg.smolsky@gmail.com>

Extension to aggregate initialization

I. Introduction

C++ supports aggregate initialization, thus allowing the following succinct notation:

```
struct user
{
    uint32_t id_;
    std::string name_;
};

user u1{10, "Alice"};
```

The Standard states that it is possible to initialize instances of such type but places numerous restrictions (such as "no base classes"). This paper argues for an amendment to relax the rules and put more power (and syntax) into the users' hands.

II. Revision history

The original paper, [N4404](#) was discussed in Lenexa and the feedback was generally positive. Several EWG members expressed a strong desire to further extend the functionality, thus prompting another revision.

III. Motivation

There are several cases revolving around inheritance of simple aggregate types that frequently come up:

1. Inheriting from a default-constructible base class.

Comes up when using template-based libraries such as Boost [StateChart](#). The following is needed to define a state with a single member:

```
struct status_event : boost::statechart::event
{
    status_event(uint32 seq) : seq_(seq) {}    // manually written boiler plate
    uint32 seq_;
};
```

The goal here is to construct a `status_event` instance while writing no boiler plate. So, let's imagine that the base class is the first, very special element of the aggregate:

```
status_event event{{}, 42};
```

2. Inheriting from an aggregate type.

Comes up when people choose to inherit from an aggregate (eg C) type:

```
struct base
{
    uint32 ibase_;
};

struct derived : base
{
    void f();
};
```

We want to construct a `derived` instance while still explicitly initializing the base class:

```
derived d{{42}};
```

As per 8.5.1 [dcl.init.aggr], it is possible to initialize instances of simple structs and there are provisions for member structs (2), empty members (9) and even brace elision (13). However, Clause (1) states that none of this goodness works when there are base classes.

It would be really useful to extend aggregate initialization rules to cover cases where base classes are present.

IV. Scope

Lets consider the following cases:

- A single base class:

```
struct base { int a1, a2; };
struct derived : base { int b1; };

derived d1{{1, 2}, 3};      // full explicit initialization
derived d1{{}, 1};         // the base is value initialized
```

The most-derived type is aggregate-initialized here and so the value b1 is reasonably obvious. The base class must be value-initialized in such a case.

- Multiple base classes, mixing user-defined constructors with the compiler-provided ones.

```
struct base1 { int b1, b2 = 42; };
struct base2 {
    B() {
        b3 = 42;
    }
    int b3;
};
struct derived : base1, base2 {
    int d;
};

derived d1{{1, 2}, {}, 4}; // full initialization
derived d2{{}, {}, 4};    // value-initialized bases
```

The most-derived type is aggregate-initialized, while the base classes are initialized with a mix of user-provided constructors, member initializers and value initialization.

V. Technical specifications

Add a new paragraph to section 8.5.1 "Aggregates [dcl.init.aggr]", between the existing paragraphs 1 and 2:

1. The *elements* of an aggregate are:
-- for an array, the array elements in increasing subscript order
-- for a class, the direct base classes in declaration order followed by the direct members in declaration order

Edit section 8.5.1 "Aggregates [dcl.init.aggr]"

1. An *aggregate* is an array or a class (Clause 9) with no user-provided constructors (12.1), no private or protected non-static data members (Clause 11), ~~no base classes (Clause 10), and~~ no virtual functions (10.3) and only non-virtual public base classes (10.1). [Note: Aggregate initialization does not allow inaccessible members to be initialized. This applies to protected and private members as well as protected and private base classes, as the notation treats the latter like special initial elements. —end note]
2. When an aggregate is initialized by an initializer list, as specified in 8.5.4, the elements of the initializer list are taken as initializers for the ~~members~~ elements of the aggregate, in ~~increasing-subscript-or-member~~ order. Each ~~member~~ element is copy-initialized from the corresponding initializer-clause. If the initializer-clause is an expression and a narrowing conversion (8.5.4) is required to convert the expression, the program is ill-formed.

3. [no change]
4. [no change]
5. [no change]
6. [no change]
7. If there are fewer initializer-clauses in the list than there are **members elements** in the aggregate, then each **member element** not explicitly initialized shall be initialized from its brace-or-equal-initializer or, if there is no brace-or-equal-initializer, from an empty initializer list (8.5.4).
8. [no change]
9. If an aggregate class C contains a subaggregate **member element m e** that has no members for purposes of aggregate initialization, the initializer-clause for **m e** shall not be omitted from an initializer-list for an object of type C unless the initializer-clauses for all members of C following **m e** are also omitted.
10. [no change]
11. [no change]
12. Braces can be elided in an initializer-list as follows. If the initializer-list begins with a left brace, then the succeeding comma-separated list of initializer-clauses initializes the **members elements** of a subaggregate; it is erroneous for there to be more initializer-clauses than **members elements**. If, however, the initializer-list for a subaggregate does not begin with a left brace, then only enough initializer-clauses from the list are taken to initialize the **members elements** of the subaggregate; any remaining initializer-clauses are left to initialize the next **member element** of the aggregate of which the current subaggregate is a **member element**.
13. All implicit type conversions (Clause 4) are considered when initializing the aggregate member with an assignment-expression. If the assignment-expression can initialize a **member element**, the **member element** is initialized. Otherwise, if the **member element** is itself a subaggregate, brace elision is assumed and the assignment-expression is considered for the initialization of the first **member element** of the subaggregate. [*Note:* As specified above, brace elision cannot apply to subaggregates with no **members elements** for purposes of aggregate initialization; an initializer-clause for the entire subobject is required. —*end note*]
14. *Note:* An aggregate array or an aggregate class may contain **members elements** of a class type with a user-provided constructor (12.1). Initialization of these aggregate objects is described in 12.6.1. —*end note*]

VI. Acknowledgments

I want to thank Richard Smith for the early feedback as well as enormous specification help.